

Cryptography — Timestamped Secrets

Attaque sur PRNG prédictible basé sur un Timestamp Unix (AES-128-ECB)

PLATEFORME : picoCTF 2026 CATÉGORIE : Cryptography DIFFICULTÉ / AUTEUR : Medium / Yahaya Meddy

FLAG : picoCTF{sa3S_sEc9t_194672d0}

//# 1. OBJECTIF & RÉSUMÉ DE LA VULNÉRABILITÉ

L'objectif du challenge est de déchiffrer un message intercepté chiffré en AES-128-ECB. La vulnérabilité découle d'une ****mauvaise génération de clé cryptographique**** : la clé symétrique dépend uniquement d'une valeur prédictible (le timestamp Unix d'exécution).

CONCEPT CLÉ CYBERSÉCURITÉ : FAIBLESSE D'ENTROPIE & PKCS#7 PADDING

Un générateur de clé dérivé du temps système (`int(time.time())`) réduit drastiquement l'espace de recherche (entropie). L'apparition de caractères comme ` ` au déchiffrement correspond au bourrage **PKCS#7**, utilisé pour aligner la taille du texte sur des blocs de 16 octets.

//# 2. OUTILS & CONCEPTS CLÉS

Outil / Concept	Rôle & Utilisation dans le CTF
CyberChef	Plateforme de manipulation de données (reconstruction de la recette From Hex + AES Decrypt).
AES-128-ECB	Mode de chiffrement par bloc sans vecteur d'initialisation (IV), vulnérable si la clé est faible.
SHA-256 (Tronqué)	Fonction de hachage appliquée à la représentation textuelle du timestamp Unix.
PKCS#7 Padding	Mécanisme complétant le dernier bloc avec des octets dont la valeur égale le nombre d'octets ajoutés (ici 14 octets <code>����</code>).

//# 3. DÉMARCHE PAS À PAS (MÉTHODOLOGIE)

Étape 1 : Analyse du Code Source Python

Le script d'encryption révèle la logique exacte de génération de la clé :

```
timestamp = int(time.time())
key = sha256(str(timestamp).encode()).digest()[:16]
cipher = AES.new(key, AES.MODE_ECB)
```

La clé correspond aux 16 premiers octets bruts (ou 32 caractères hexadécimaux) du SHA-256 du timestamp Unix.

Étape 2 : Dérivation de la Clé avec le Timestamp fourni

L'indice indique que le chiffrement a été réalisé autour du timestamp `1770242597`.

- String d'entrée : `"1770242597"`
- Hash SHA-256 complet : `1e072065e84dd5...`

- Clé AES Hex (32 hex / 16 bytes) : `1e072065e84dd5...` (formatée en **HEX** dans CyberChef)

Étape 3 : Déchiffrement dans CyberChef

Configuration de la recette CyberChef :

1. **From Hex** (pour convertir le Ciphertext `77c36bef...` en octets)
2. **AES Decrypt** :
 - Mode : **ECB/NoPadding**
 - Key : `1e072065e84dd5...` (Input Format: **HEX**)

Résultat obtenu :

```
picoCTF{sa3S_sEc9t_194672d0}
```

//# 4. LEÇONS RETENUES & REFLEXES À GARDER

AUTOMATISMES DE SÉCURITÉ CRYPTOGRAPHIQUE

- **Formats de clés dans les outils** : En cryptographie pratique, toujours s'assurer du format d'entrée des clés (brut, UTF-8, Base64 ou Hexadecimal). Une clé binaire doit être déclarée en format **HEX** dans des utilitaires comme CyberChef.
- **Ne jamais utiliser le temps comme source d'entropie** : Utiliser des générateurs cryptographiquement sûrs (ex: `secrets` ou `os.urandom()` en Python) plutôt que `time.time()` ou `random`.
- **Comprendre le Bourrage (Padding)** : Ne pas s'inquiéter des octets non imprimables en fin de chaîne déchiffrée : ils font partie du mécanisme normal de bourrage PKCS#7.